M13 # Worktrees

Methodology Advanced

Work in parallel on multiple branches without switching directories

Principle

A git worktree creates a second (or third) working directory from the same repository, each checked out on a distinct branch. No `git stash`, no branch switching: two terminals, two contexts, zero friction between them.

Available since Git 2.5.0 (2015), natively compatible with Claude Code.

Essential commands

```
# Create a worktree on an existing or new branch
git worktree add ../myproject-feature feature/auth
git worktree add ../myproject-hotfix -b hotfix/login-bug

# List all active worktrees
git worktree list

# Clean up after merge
git worktree remove ../myproject-hotfix
git worktree prune   # Remove stale references
```

Parallel Claude Code pattern

```
# Terminal 1: main feature
cd ../myproject-feature
claude
> "Implement JWT authentication"

# Terminal 2: urgent hotfix (simultaneous)
cd ../myproject-hotfix
claude
> "Fix the timeout bug on Safari"
```

Each Claude instance indexes its own worktree and maintains an independent context. Files are separated, but ports, databases and external services may still be shared.

Claude Code context in worktrees

File	Scope
`~/.claude/CLAUDE.md`	Global, shared by all worktrees
`CLAUDE.md` (repo root)	Project, committed, shared
`CLAUDE.local.md` (add to `.gitignore`)	Personal instructions in this worktree

A worktree can have its own `.claude/` configuration for branch-specific conventions.

Dependency management

The main limitation: `node_modules` or `vendor/` are not shared automatically. Each worktree needs its own dependencies, which can take disk space and time.

Recommended approach: install dependencies independently in each worktree, using the package manager cache to reduce downloads. A shared `node_modules` symlink couples branches and is unsafe when lockfiles or install state differ.

```
# From the new worktree, using the repository's package manager
pnpm install --frozen-lockfile
```

Review any custom `/git-worktree` helper before using its dependency-sharing options.

When to use worktrees

Suited for: multiple features in parallel, comparing approaches, code review during development, long CI builds while continuing to code.

Not suited for: simple and quick branch switches, teams unfamiliar with the tool (adds operational complexity), limited disk space.

Shell aliases for quick navigation

```
# In ~/.zshrc
alias wa="cd ../myproject-feature-a"
alias wb="cd ../myproject-feature-b"
alias wm="cd ../myproject"   # Main worktree
```

Pattern used by the Claude Code team internally to navigate instantly between active worktrees.

worktree.baseRef (v2.1.133)

Controls the base commit used when creating a worktree via `--worktree`, `EnterWorktree`, or agent isolation.

Value	Behavior
`fresh` (default)	Branch from `origin/<default-branch>`, always a clean remote base
`head`	Branch from local HEAD, includes unpushed commits

Breaking change v2.1.133: `fresh` is now the default. If you have unpushed commits that need to be in the worktree branch, add this to `settings.json`:

```
{
  "worktree": {
    "baseRef": "head"
  }
}
```

Cleanup after merge

After merging the branch, remove the worktree to free up space. `git worktree prune` cleans references to worktrees that were manually deleted (folder removed without going through `git worktree remove`).